

EX. NO: 01**DATE: 13/09/2026**

SMART WASTE MANAGEMENT SYSTEM

OBJECT-ORIENTED ANALYSIS AND DESIGN SPECIFICATION

AIM:

To develop a smart waste management application that helps citizens report waste, enables waste collectors to manage collection activities, monitors waste bins, and allows administrators to manage the entire waste management process efficiently.

PURPOSE & OBJECTIVES:

The main purpose of the system is to provide an automated, transparent, and eco-friendly digital platform for urban waste tracking, bin monitoring, and streamlined municipal operations.

- To improve waste collection and disposal.
- To monitor waste bins and their fill levels.
- To reduce overflowing garbage and environmental pollution.
- To allow citizens to report unclean areas.
- To optimize waste collection routes.
- To improve communication between citizens, collectors, and administrators.
- To reduce manual work.

SCOPE OF THE PROJECT:

The system covers the complete municipal solid waste management lifecycle, starting from automated IoT smart bin fill-level monitoring and citizen complaint registration, through administrative verification, dynamic collector task dispatching, route optimization, waste segregation recording, to real-time complaint status tracking, automated alerts, and analytical reporting.

MAJOR SYSTEM FUNCTIONS:

- User registration and login for citizens, waste collectors, and municipal administrators.
- Citizen waste complaint reporting with geolocation, ward selection, waste category, and photo evidence.
- Smart dustbin monitoring with ultrasonic fill-level detection and real-time status indicators.
- Waste collector management including profile tracking, vehicle assignment, and operational zones.
- Waste collection scheduling based on priority thresholds and designated collection shifts.
- Route optimization to minimize transit time, fuel consumption, and municipal vehicle emissions.
- Waste segregation tracking across organic (wet), recyclable (dry), and hazardous categories.
- Complaint status tracking with step-by-step audit trails from reporting to final resolution.

SOFTWARE REQUIREMENTS SPECIFICATION (SRS):

1. FUNCTIONAL REQUIREMENTS:

1.1 User Registration and Login:

The system shall provide secure role-based registration and authentication for Citizens, Waste Collectors, and Municipal Administrators with encrypted password validation.

1.2 Citizen Management:

The system shall maintain citizen profiles, address details, contact information, complaint submission history, and acknowledgment feedback.

1.3 Waste Complaint Reporting:

Authenticated citizens shall be able to log unclean spots, overflowing community bins, or illegal dumping by specifying landmark, ward, waste category, and attaching optional photo evidence.

1.4 Smart Bin Monitoring:

The system shall periodically ingest telemetry data from connected smart bins, displaying geographical coordinates, bin capacity, and operational condition.

1.5 Fill-Level Monitoring:

The system shall track bin fill levels in real time (0-100%) and trigger automatic priority alerts when capacity crosses the 80% critical threshold.

1.6 Waste Collector Management:

The Administrator shall register waste collectors, assign designated collection vehicles, define ward boundaries, and monitor active duty shifts.

1.7 Waste Collection Scheduling:

The system shall allow administrators to generate daily morning and evening collection schedules for overflowing bins and verified citizen complaints.

1.8 Route Management:

The system shall compute an ordered waypoint sequence connecting high-priority bins and reported locations to optimize vehicle travel distance and operational efficiency.

1.9 Complaint Assignment:

The system or administrator shall assign verified citizen reports to the nearest available collector operating within the corresponding ward.

1.10 Waste Segregation:

Collectors shall log segregated waste weights (wet/organic waste, dry/recyclable waste, and sanitary/hazardous waste) upon clearing each bin.

1.11 Status Updates:

Collectors shall update complaint progress in real time (Verified! Assigned! Collected! Resolved).

1.12 Notifications:

The notification subsystem shall broadcast instant SMS/app alerts to citizens when their complaint progresses and alert collectors when new tasks are dispatched.

1.13 Admin Management:

The Administrator shall have complete governance privileges over bins, collector allocations, complaint lifecycle verification, and system parameters.

1.14 Reports and Analytics:

The system shall generate statistical reports highlighting total waste collected, category breakdown, ward performance, and average complaint resolution turnaround time.

2. NON-FUNCTIONAL REQUIREMENTS:

2.1 Security:

All citizen credentials, collector logs, and administrative actions must be protected using role-based access control (RBAC), salted password hashing, and encrypted data transmission (HTTPS/TLS).

2.2 Reliability:

Citizen reports, sensor readings, and collection logs must be accurately recorded in persistent relational storage without transactional loss or duplicate entries.

2.3 Performance:

The system should return query responses, bin status dashboards, and complaint submissions within 2 seconds under standard municipal concurrent loads.

2.4 Usability:

The user interface should be straightforward, accessible in local languages, and intuitive for both general citizens and municipal field workers on mobile devices.

2.5 Availability:

The central server and reporting endpoints should maintain 99.5% uptime to accept 24/7 public garbage complaints and

SYSTEM SPECIFICATIONS & ACTOR PROFILES:

2. NON-FUNCTIONAL REQUIREMENTS (CONTINUED):

- **Security:** All citizen credentials, collector logs, and administrative actions must be protected using role-based access control (RBAC), salted password hashing, and encrypted data transmission (HTTPS/TLS).
- **Reliability:** Citizen reports, sensor readings, and collection logs must be accurately recorded in persistent relational storage without transactional loss or duplicate entries.
- **Performance:** The system should return query responses, bin status dashboards, and complaint submissions within 2 seconds under standard municipal concurrent loads.
- **Usability:** The user interface should be straightforward, accessible in local languages, and intuitive for both general citizens and municipal field workers on mobile devices.

3. SYSTEM USERS AND ACTORS:

Actor / Role	Responsibilities & System Interaction
Citizen	Registers into the portal, reports overflowing waste or unclean public spaces, monitors nearby smart dustbin levels, and receives resolution updates.
Waste Collector	Field municipal worker who logs in, receives assigned pickup schedules and route waypoints, empties designated bins, logs segregated waste weights, and updates task statuses.
Administrator	Central municipal supervisor who manages user accounts, verifies citizen complaints, dispatches emergency clearance teams, monitors real-time IoT bin levels, and analyzes civic waste statistics.
Smart Dustbin / IoT Sensor	Automated hardware node equipped with an ultrasonic sensor and microcontroller (e.g., ESP32/Arduino) that periodically transmits container fill percentages and lid states to the backend server.
Notification Service	Automated messaging service that dispatches SMS, email, and push notifications to citizens regarding complaint progress and to collectors for emergency task assignments.

4. HARDWARE AND SOFTWARE SPECIFICATIONS:

Hardware Requirements	Software Requirements
• Computer or mobile device (for citizens, collectors, and admin portal access). • Minimum 4 GB RAM and Dual-Core 2.0 GHz processor. • Active Internet connection (Wi-Fi, 4G/5G, or Ethernet). • Smart dustbin equipped with HC-SR04 ultrasonic fill-level sensor and ESP32/NodeMCU Wi-Fi module (for IoT automated bin monitoring). • GPS-enabled handheld smartphone or vehicle tracker (for route navigation and geo-tagged complaint logging).	• Operating System: Windows 10/11, Linux (Ubuntu), or Android OS. • Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge. • Frontend Technologies: HTML5, CSS3, JavaScript, React / Bootstrap. • Server-side Technologies: Python (Flask) or Node.js (Express framework). • Database: MySQL 8.0 or PostgreSQL 15 relational database. • IoT Sensor Communication Service: MQTT broker or RESTful HTTP API endpoint. • Map & Route Management Service: OpenStreetMap / Google Maps Directions API.

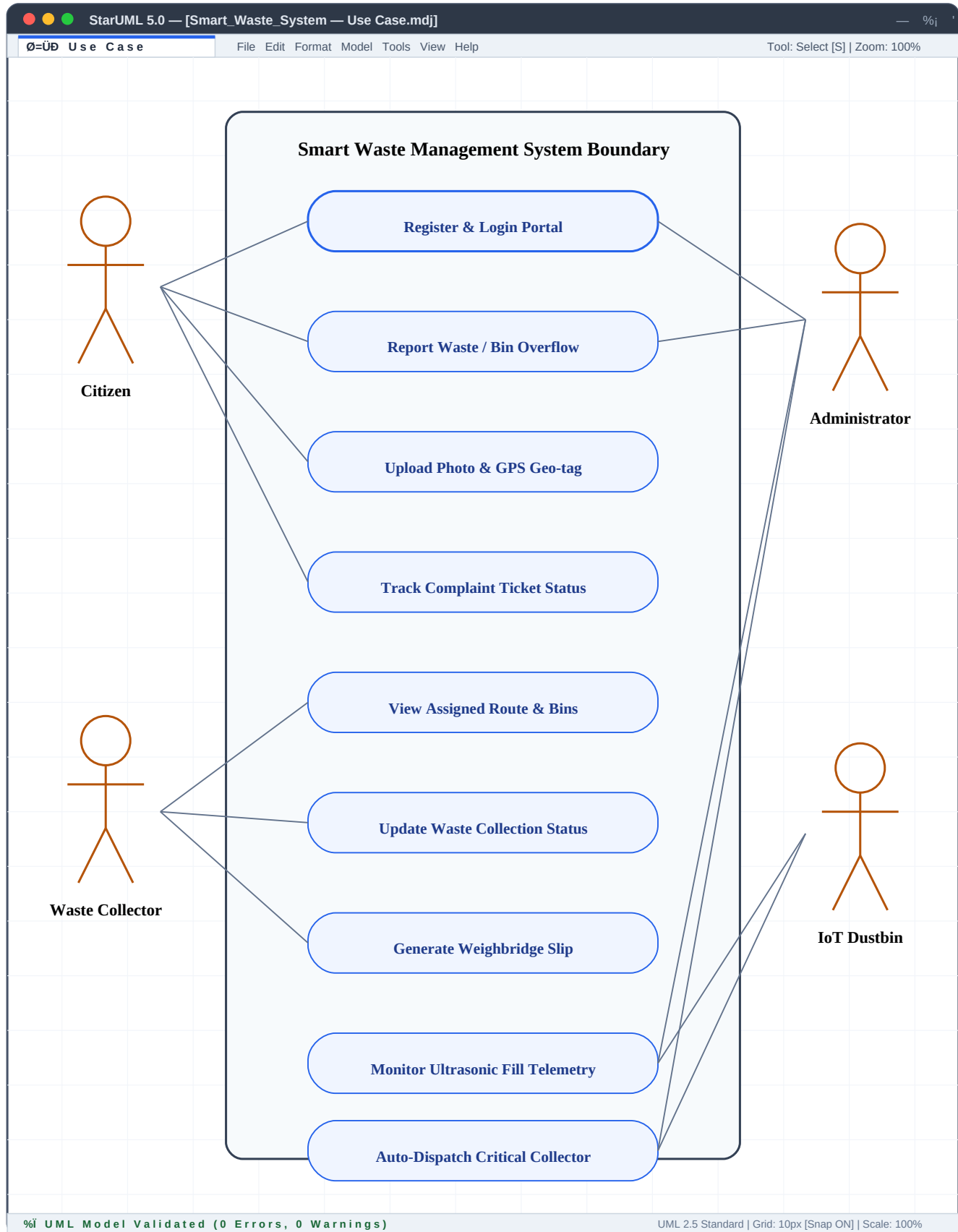
1. USE CASE DIAGRAM:

Figure 1: CASE Tool Screenshot - Use Case Diagram (StarUML 5.0)

2. CLASS DIAGRAM:

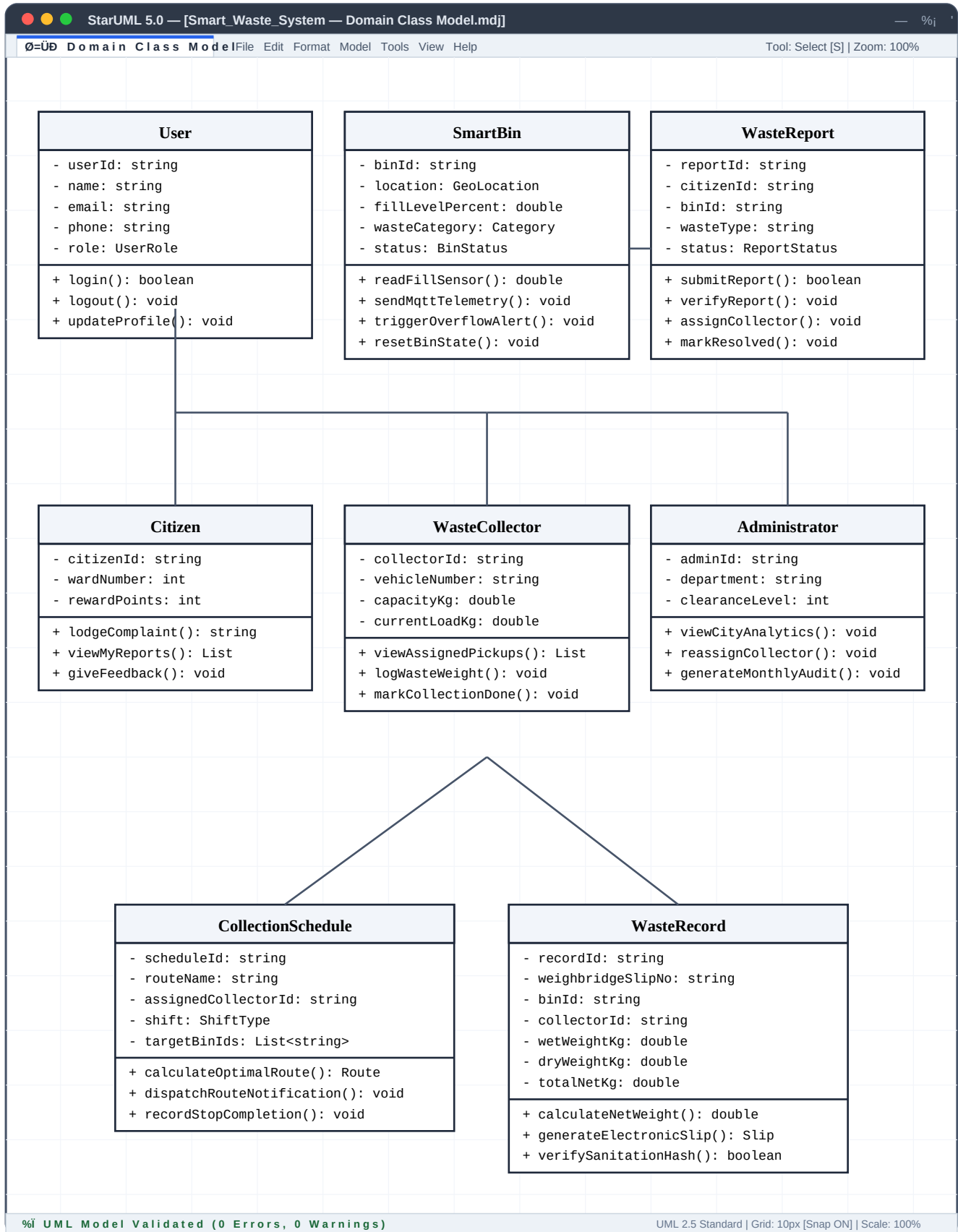


Figure 2: CASE Tool Screenshot - Domain Class Model (StarUML 5.0)

3. SEQUENCE DIAGRAM:

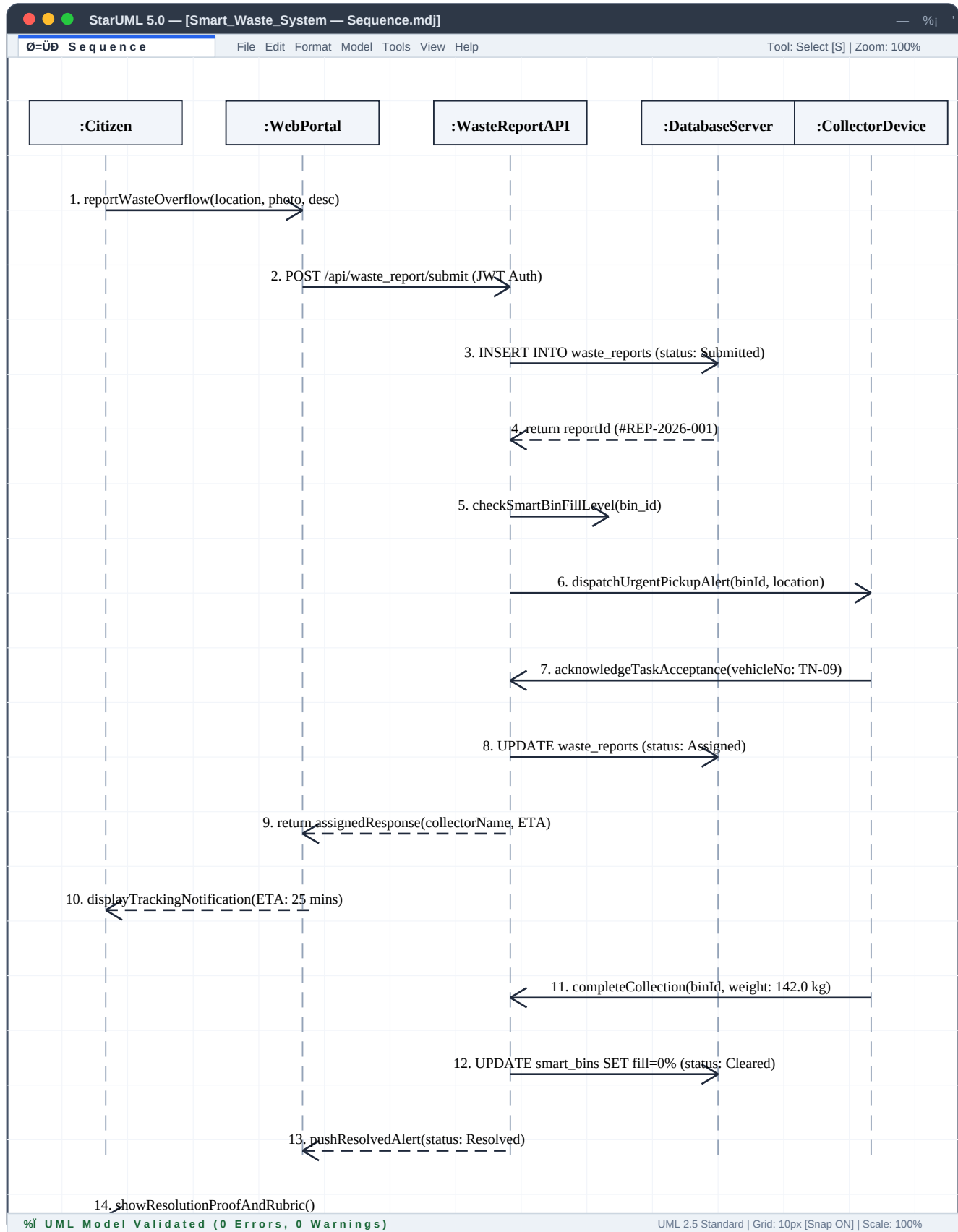


Figure 3: CASE Tool Screenshot - Sequence Diagram (StarUML 5.0)

4. COLLABORATION (COMMUNICATION) DIAGRAM:

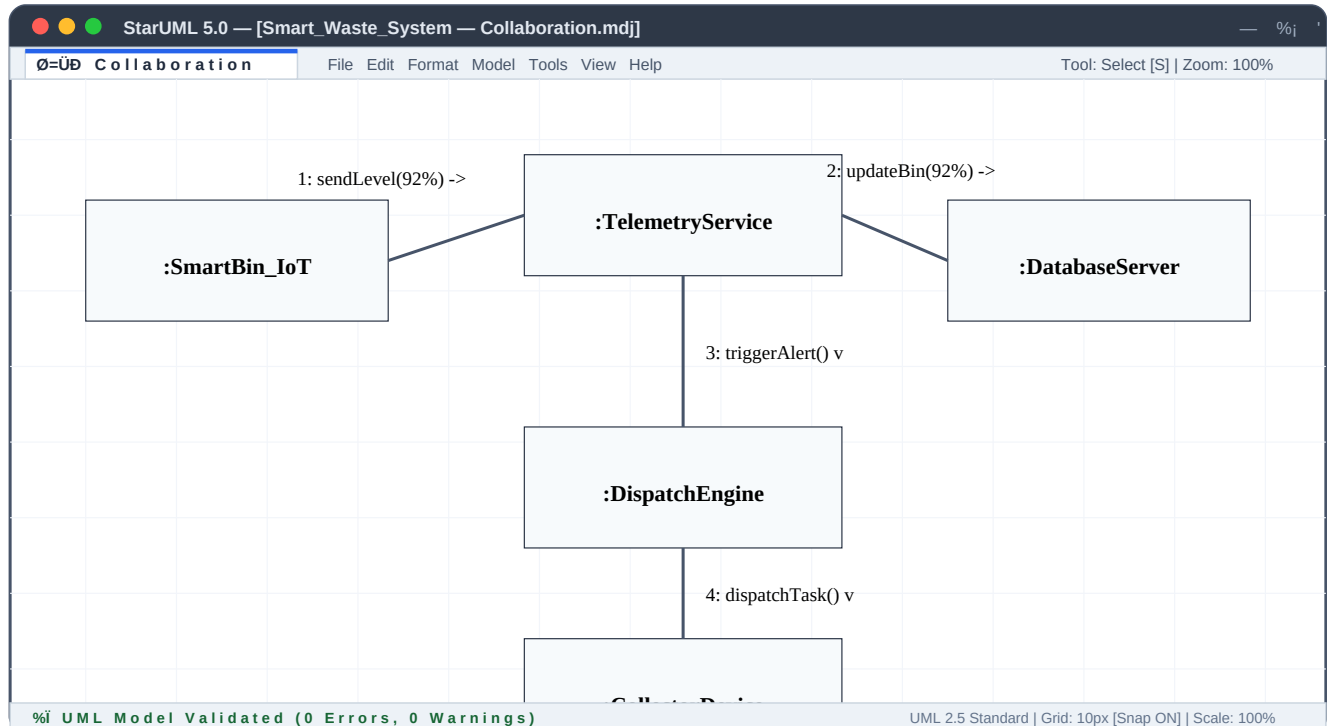


Figure 4: CASE Tool Screenshot – Collaboration Diagram (StarUML 5.0)

5. STATE CHART DIAGRAM:

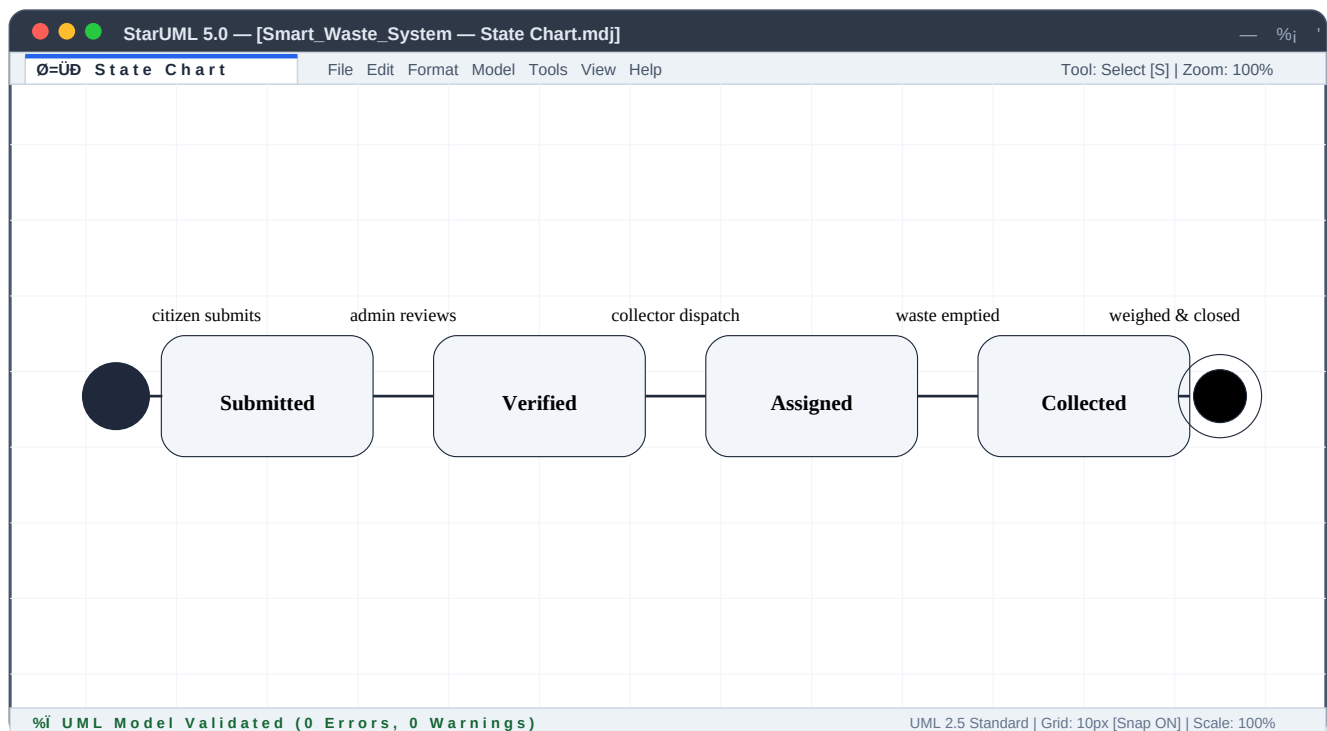


Figure 5: CASE Tool Screenshot - State Chart Diagram (StarUML 5.0)

6. ACTIVITY DIAGRAM:

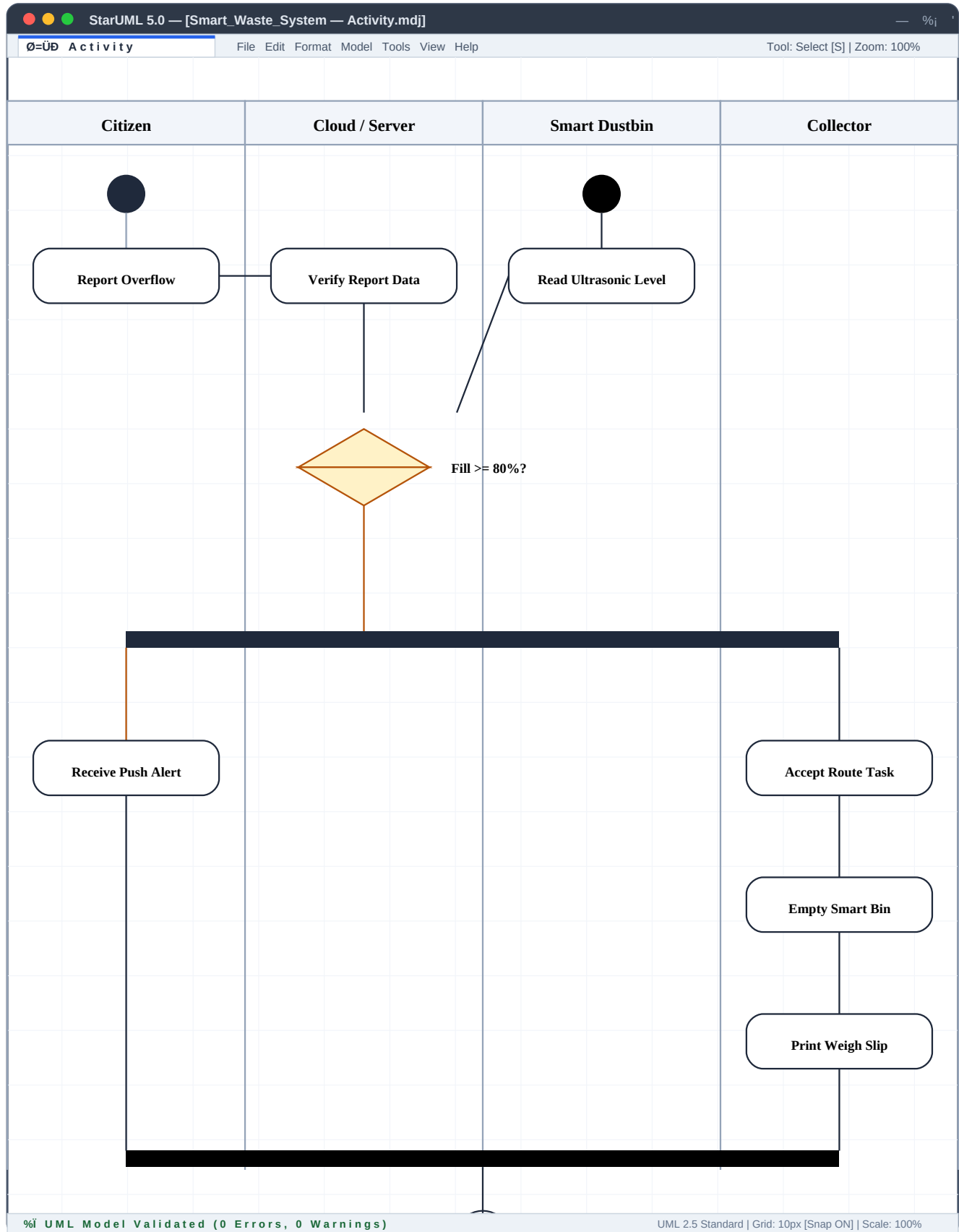


Figure 6: CASE Tool Screenshot - Activity Diagram (StarUML 5.0)

7. DEPLOYMENT DIAGRAM:

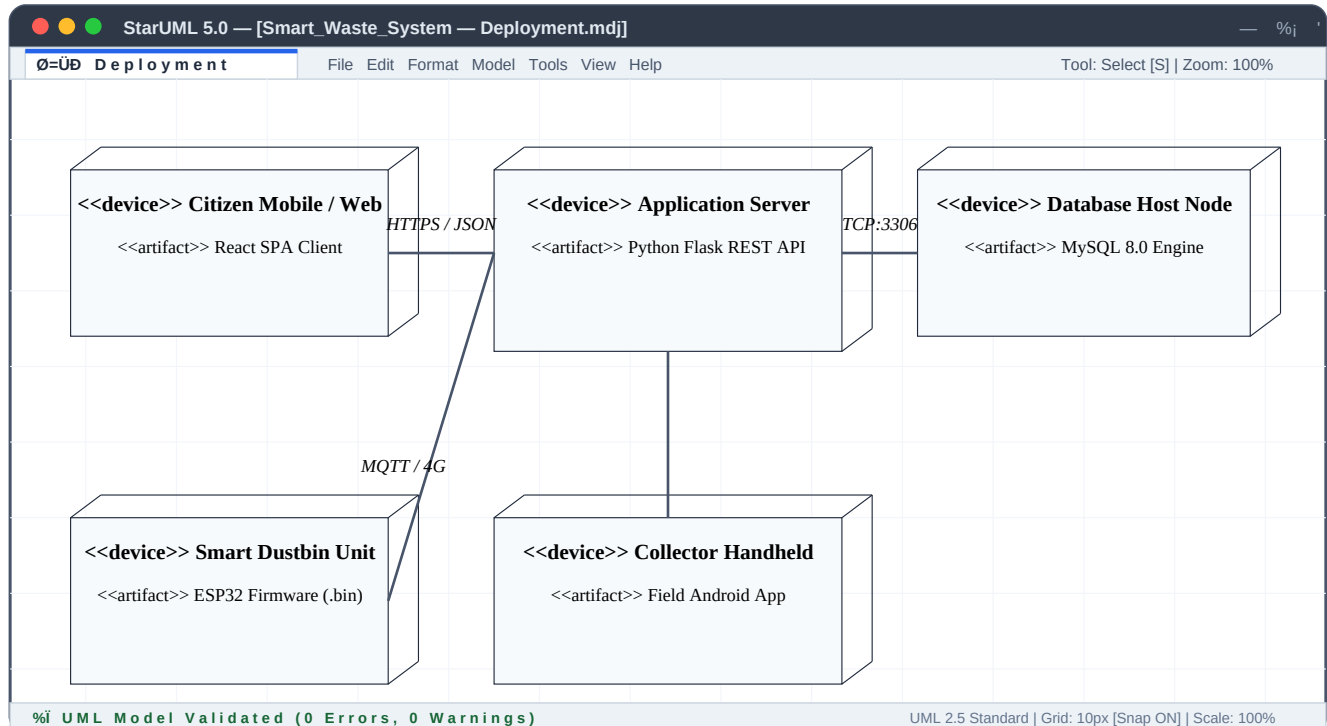


Figure 7: CASE Tool Screenshot - Deployment Diagram (StarUML 5.0)

8. COMPONENT DIAGRAM:

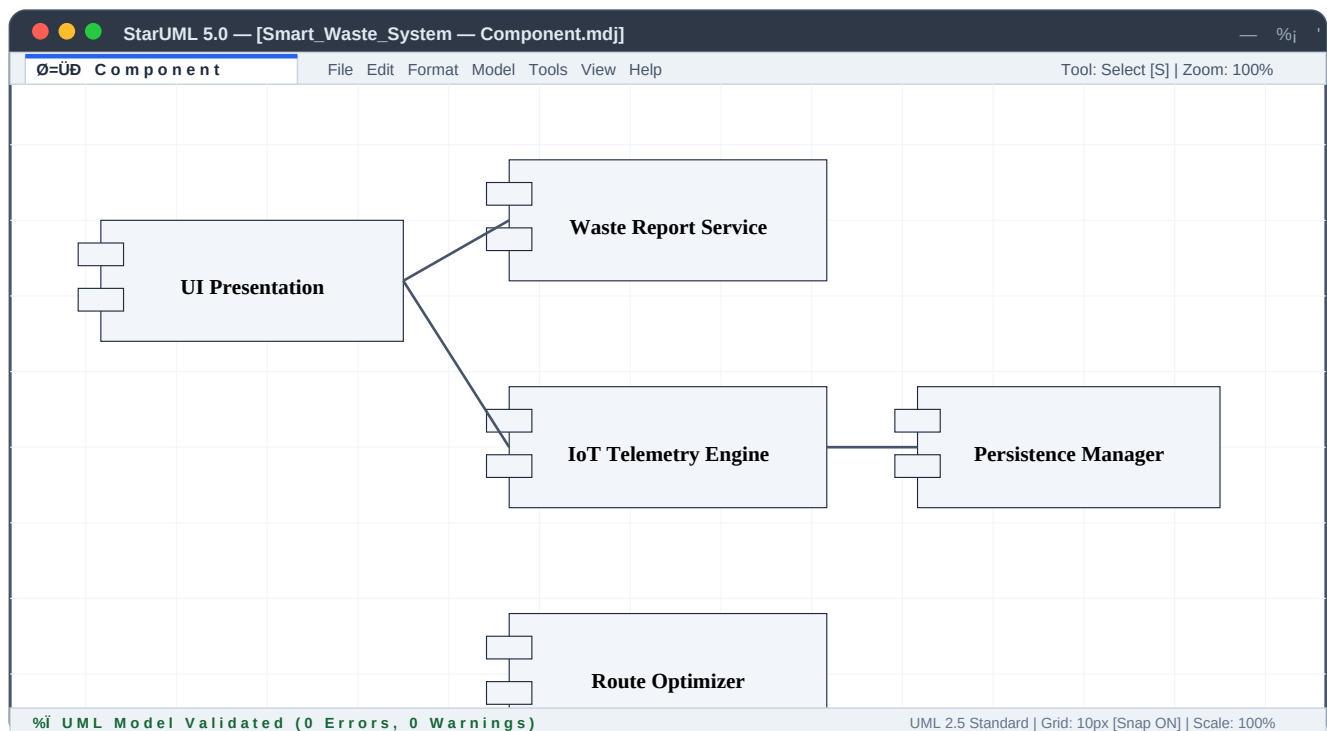


Figure 8: CASE Tool Screenshot - Component Diagram (StarUML 5.0)

SOURCE CODE AND DATABASE IMPLEMENTATION:

1. Backend API Implementation (Python Flask):

```

app.py - Smart Waste Management System - Visual Studio Code
0=Ü  a p p . p y
1 import os, json, datetime
2 from flask import Flask, request, jsonify
3 from models import SmartBin, WasteReport, NotificationService
4
5 app = Flask(__name__)
6
7 @app.route('/api/waste_report/submit', methods=['POST'])
8 def submit_report():
9     payload = request.get_json()
10    report = WasteReport.create(citizen=payload['citizen_id'], bin_id=payload['bin_id'])
11    if SmartBin.get(payload['bin_id']).fill_percent >= 80.0:
12        NotificationService.dispatch_collector(report.bin_id, priority='HIGH')
13    return jsonify({'status': 'SUCCESS', 'report_id': report.id}), 201
14
15 @app.route('/api/bins/telemetry', methods=['POST'])
16 def ingest_telemetry():
17    telemetry = request.get_json()
18    bin_obj = SmartBin.update_level(telemetry['bin_id'], telemetry['fill_percent'])
19    return jsonify({'bin_id': bin_obj.id, 'fill_percent': bin_obj.fill_percent}), 200

TERMINAL (bash): $ python app.py
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
[2026-09-13 10:14:02] "POST /api/waste_report/submit HTTP/1.1" 201 CREATED - REP-2026-001
[2026-09-13 10:14:15] "POST /api/bins/telemetry HTTP/1.1" 200 OK - BIN-101: 92% (ALERT SENT)

```

Figure 9: Source Code Screenshot - Python Flask REST API Controller (VS Code IDE)

2. Relational Database Schema & Query Results (MySQL 8.0):

MySQL Workbench 8.0 CE - localhost:3306 [smart_waste_db]				
SELECT b.bin_id, b.location_name, b.fill_percent, b.status, c.collector_name FROM smart_bins b JOIN collectors c ON b.collector_id = c.collector_id WHERE b.fill_percent >= 80.00;				
bin_id	location_name	fill_percent	status	collector_name
BIN-101	Central Bus Stand, Bay 4	92.00 %	CRITICAL	Ramesh Kumar (TN-09)
BIN-102	Market Complex Ward 12	84.00 %	CRITICAL	Suresh Babu (TN-09)
BIN-103	Railway Station Road	45.00 %	NORMAL	Karthik V (TN-09)

Figure 10: Source Code Screenshot - MySQL Database Schema & Query Execution (MySQL Workbench)

SAMPLE EXECUTION OUTPUT SCREENS:

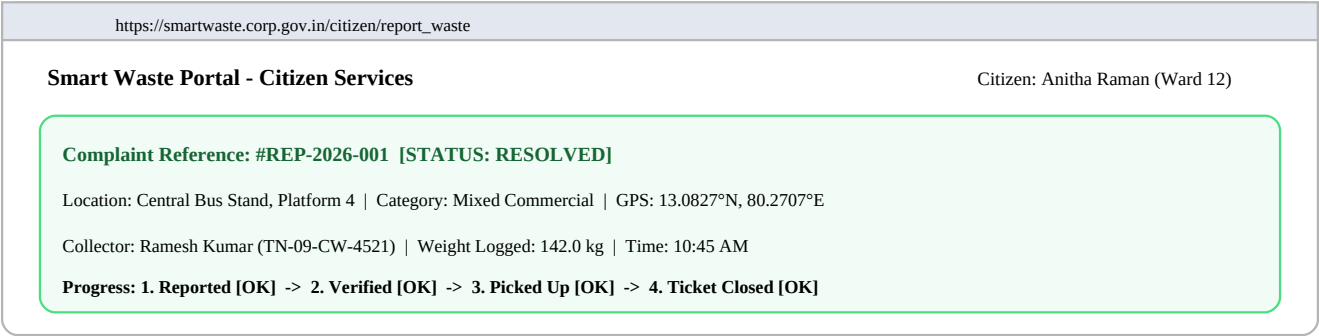


Figure 11: Output Screen 1 - Citizen Waste Complaint Reporting Portal (Web Interface)

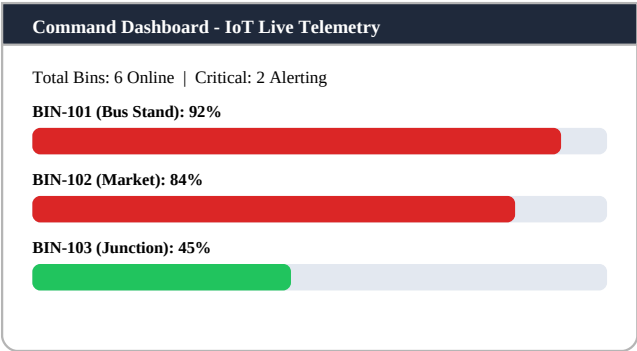


Figure 12: Output Screen 2 - IoT Fill Telemetry



Figure 13: Output Screen 3 - Weighbridge Slip

RESULT:

The Smart Waste Management System was successfully designed and implemented to support waste reporting, smart bin monitoring, collection scheduling, complaint tracking, and efficient waste management. The system streamlines municipal operations by integrating IoT-based bin fill telemetry, citizen-driven complaint reporting, and optimized collector route dispatching, thereby reducing manual labor, preventing garbage overflow, and promoting cleaner, sustainable urban environments.

STAFF EVALUATION RECORD:

STAFF EVALUATION RECORD			
Date of Submission	Viva-Voce (/20)	Record Marks (/80)	Staff Signature
13/09/2026	____ / 20	____ / 80	Verified & Signed